

MATH0499: Dinic Algorithm

Sam EL MASRI, SeyedPouria SALEHI KATOZI

04/12/2021



Table des matières

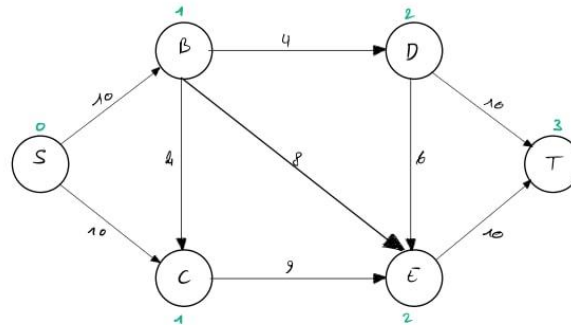
1	Introduction :	3
2	Formalisation du problème :	3
3	Analyse du problème et découpe en sous-problèmes :	4
3.1	Créer un graphe (SP1.0) :	4
3.1.1	Créer un graphe depuis un fichier (SP1.1) :	4
3.1.2	Créer un graphe aléatoire (SP1.2) :	5
3.2	Trouver tous les chemins possibles (SP2) :	5
3.3	Calculer la capacité minimale dans un chemin (SP3) :	5
3.4	Calculer le flot de chaque chemin et modifier le graphe (SP4) :	5
3.5	Flot maximal (SP5) :	5
4	Résolution des sous-problèmes :	5
4.1	Résolution SP1 :	5
4.1.1	Résolution SP1.1 :	5
4.1.2	Résolution SP1.2 :	5
4.2	Résolution SP2 :	5
4.3	Résolution SP3 :	6
4.4	Résolution SP4 :	6
4.5	Résolution SP5 :	6
5	Les challenges et les difficultés rencontrés :	6
5.1	Initialisation et création du graphe :	6
5.2	Trouver un "augmenting level path" :	6
5.3	Faire des opérations sur les arcs :	6
5.4	La programmation défensive :	6
6	Conclusion :	7
7	Comment utiliser le programme :	7
8	Les sources :	7

1 Introduction :

L'objectif principal de ce projet est d'appliquer l'algorithme de Dinic (ou Dinitz) qui nous aide à calculer le flot maximal dans un graphe donné. Cet algorithme a été inventé en 1970 par l'informaticien Yefim Dinitz.

2 Formalisation du problème :

Nous avons un graphe orienté, chaque sommet de notre graphe possède un niveau et chaque arc une capacité.



Nous devons calculer le flot maximal entre 2 sommets, le sommet de **départ (source)** et le sommet **d'arrivée (sink)** en vérifiant que le niveau de chaque sommet qu'on parcourt est supérieur au sommet précédant dans le chemin qu'on a choisi pour arriver à sommet **arrivée (sink)**.

On calcule le flot maximal d'un chemin grâce à la capacité de chaque arc et la capacité minimale de tous les arcs dans le chemin qu'on a choisi.

Dans l'exemple au dessus, on procède comme suite et notre graphe devient :

chemin : $S \rightarrow B \rightarrow D \rightarrow T$

niveaux : $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$

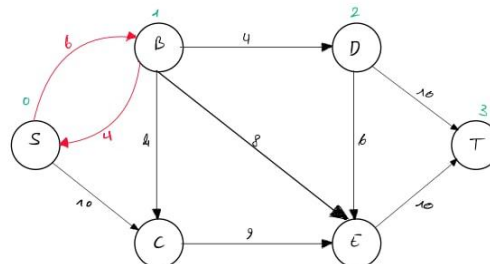
Comme l'ordre de niveau de chaque sommet est strictement croissant de 1 alors le chemin choisi est correcte et valide donc on peut continuer à appliquer l'algorithme.

Capacité minimale : 4

$S \rightarrow B$

$\text{flot}(S, B) = 10 - 4 = 6$

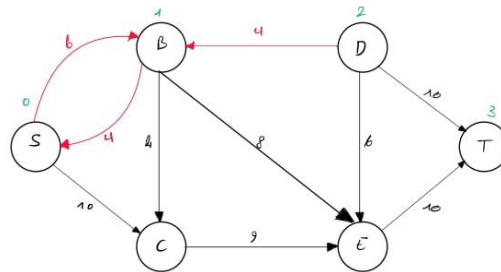
$\text{flot}(B, S) = 4 + 0 = 4$



B → D

$$\text{flot}(\text{B}, \text{D}) = 4 - 4 = 0$$

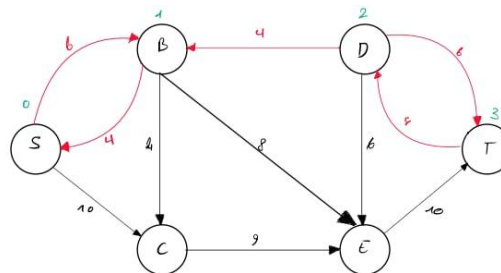
$$\text{flot}(\text{D}, \text{B}) = 0 + 4 = 4$$



D → T

$$\text{flot}(\text{D}, \text{T}) = 10 - 4 = 6$$

$$\text{flot}(\text{T}, \text{D}) = 0 + 4 = 4$$



Nous ne pouvons plus calculer le flot de chemin $S \rightarrow B \rightarrow D \rightarrow T$ puisque les niveaux des sommet sont changé respectivement à $0 \rightarrow 2 \rightarrow 1 \rightarrow 2$ et ils ne sont pas d'ordre croissant de 1.

Nous sommes le flot maximal de tous les chemins possible entre le sommet de **départ (source)** et le sommet d'**arrivée (sink)** en vérifiant que les chemins sont corrects pour trouver le flot maximal de notre graphe.

3 Analyse du problème et découpe en sous-problèmes :

Nous pouvons diviser le problème principal d'algorithme en 4 sous-problèmes, mais comme nous devons créer un graphe alors nous avons principalement 5 sous-problèmes :

3.1 Créer un graphe (SP1.0) :

3.1.1 Créer un graphe depuis un fichier (SP1.1) :

Nous allons lire un fichier contenant toutes les informations nécessaires pour créer notre graphe.

3.1.2 Créer un graphe aléatoire (SP1.2) :

Nous pouvons également créer un graphe d'une manière aléatoire en demandant à l'utilisateur le nombre de sommets.

3.2 Trouver tous les chemins possibles (SP2) :

Tout d'abord, il nous faut trouver un chemin possible entre le sommet de **départ (source)** et le sommet **d'arrivée (sink)** en vérifiant que ce chemin est valide (chemin le plus court possible). Autrement dit un chemin où tout les niveaux des sommets sont strictement croissants. Les deux sommets peuvent être choisis par l'utilisateur.

3.3 Calculer la capacité minimale dans un chemin (SP3) :

Nous avons expliqué précédemment dans la partie "Formalisation du problème" que chaque arc possède une capacité. Nous devons alors chercher la capacité minimale de tous les arcs présents dans un chemin donné (chemin le plus court possible).

3.4 Calculer le flot de chaque chemin et modifier le graphe (SP4) :

Pour calculer le flot entre 2 sommets A et B, on procède comme suit :

$\text{flot}(A,B) = [\text{La capacité d'arc qui relie A et B}] - [\text{La capacité minimale du chemin}]$

$\text{flot}(B,A) = [\text{La capacité minimale du chemin}] + [\text{La capacité d'arc qui relie B et A}]$

Après avoir calculé les flots on modifie le graphe et on remplace les nouveaux flots par la capacité d'arc qui était entre les 2 sommets A et B. A termes, la source devrait être isolée du puits.

3.5 Flot maximal (SP5) :

La somme de toutes les capacités minimales (SP3) donne le flow maximal du réseau de noeuds.

4 Résolution des sous-problèmes :

4.1 Résolution SP1 :

4.1.1 Résolution SP1.1 :

Nous avons créé 2 fonctions "read_create()" et "windowed_graph()" qui nous permettent de lire le contenu d'un fichier de créer un graphe grâce aux informations qu'on récupère depuis ce fichier. Le nom de ce fichier doit être fourni par l'utilisateur.

4.1.2 Résolution SP1.2 :

Pour créer un graphe d'une manière aléatoire, nous avons créé une fonction qui s'appelle generate() qui prend le nombre de sommets en input comme paramètre.

4.2 Résolution SP2 :

Nous trouvons tous les chemins possibles dans la fonction principale dinic() qui prend un graphe G, un sommet de départ(source)et un sommet d'arrivée(sink)comme paramètres. Ensuite, dans cette fonction on vérifie tout d'abord s'il y a au moins un chemin possible entre le

sommet de départ(source) et le sommet d'arrivée(sink) avec la fonction `has_path()`. Après cet étape nous vérifions tous les chemins possibles avec les deux fonctions `check_augmenting_level_path()` et `augmenting_level_path()`.

4.3 Résolution SP3 :

Le capacité minimale se trouve d'une manière très intuitive. Tout d'abord, on récupère toutes les capacité du chemin considéré en utilisant la fonction `G.get_edge_data()`, on stocke les capacités des sommets dans une liste et ensuite on cherche la valeur minimale dans cette liste.

4.4 Résolution SP4 :

Dans la fonction `modify_edges()`, tout simplement on applique les formules que nous avons pour calculer les flows.

4.5 Résolution SP5 :

Le calcul de flot maximal se fait d'une manière assez rapide afin de faire appel aux fonctions que nous avons cité dans les sous-problèmes précédemment.

5 Les challenges et les difficultés rencontrés :

Nous avons rencontré plusieurs difficultés dans ce projet.

5.1 Initialisation et création du graphe :

Nous devons choisir un moyen de générer un graphe depuis un fichier quelconque. Nous avons beaucoup réfléchi jusqu'à choisir une méthode simple qui consiste à mettre la liste de tous les arcs dans un fichier "text". Ensuite, diviser chaque ligne en 3 et lire les informations en les plaçant dans un graphe NetworkX.

5.2 Trouver un "augmenting level path" :

Pour appliquer l'algorithme, il faut trouver plusieurs chemins reliant la source au puits. Suite à de nombreuses recherches, nous avons trouvé une fonction NetworkX qui permet de donner un chemin valable. Effectivement, ce chemin est bien à niveau croissant puisque le chemin est le plus court possible.

5.3 Faire des opérations sur les arcs :

L'algorithme nous oblige à soustraire et additionner les arcs par une certaine valeur. Ici le problème est lié à l'utilisation de la librairie NetworkX. Pouvoir modifier les valeurs d'un arc, etc.

5.4 La programmation défensive :

Le fait de protéger le programme de cas pathologiques. Nous avons dû consacrer beaucoup de temps sur la programmation défensive.

6 Conclusion :

Nous avons créé une fonction permettant de calculer le flot maximale dans un graphe avec une complexité temporelle quadratique. Cet algorithme était très efficace et simple pour son époque.

7 Comment utiliser le programme :

Veuillez lire le "README" disponible dans le dossier "code" du projet.

8 Les sources :

Tout d'abord nous avons consulté le wikipedia :
https://fr.wikipedia.org/wiki/Algorithme_de_Dinic
<https://en.wikipedia.org/wiki/Dinic>

Ensuite nous avons regardé 2 videos sur youtube pour être sûr de notre compréhension d'algorithme :

<https://www.youtube.com/watch?v=M6cm8UeeziI>
<https://www.youtube.com/watch?v=duKIzgJQ1w8>